

Technical University Dortmund
University of Naples Federico II

**METHODS FOR FINTECH & ARTIFICIAL INTELLIGENCE IN
FINANCE**

**Image Compression with Principal Component Analysis
(PCA): A Study on Dimensionality Reduction**

Supervisor:

Professor Maria Iannario

Author:

Sabrina Sultana

Matriculation Number : 235062

October 7, 2024

Contents

1	Introduction	1
2	Problem Statement	1
3	PCA Overview: Mathematical Foundation	2
3.1	Steps in PCA	2
3.1.1	Standardization of Data	2
3.1.2	Covariance Matrix Computation	2
3.1.3	Eigenvalue Decomposition	3
3.1.4	Selection of Components Based on Explained Variance	3
4	PCA Workflow in Image Compression	4
4.1	Read Original Image	4
4.2	Compute Principal Components	5
4.3	Forward Transform	5
4.4	Reconstruct the Compressed Image	5
5	Code Implementation of PCA in Python	6
5.1	Import Necessary Libraries	6
5.2	Load and Preprocess the Image	6
5.3	Apply PCA to the Image Data	6
5.4	Reconstruct the Image	7
6	Result & Analysis	7
7	Conclusion	9

1 Introduction

With the exponential growth of digital data in recent years, efficient storage, transmission, and processing of data, particularly images, have become critical challenges in various fields, including data science, computer vision, and multimedia applications. One of the key techniques that addresses these challenges is dimensionality reduction, where high-dimensional data is projected onto a lower-dimensional space to capture the essential patterns while reducing redundancy.

Principal Component Analysis (PCA) is to reduce the dimensionality of a data set consisting of a large number of interrelated variables, while retaining as much as possible of the variation present in the data set. This is achieved by transforming to a new set of variables, the principal components (PCs), which are uncorrelated, and which are ordered so that the first few retain most of the variation present in all of the original variables. [1]

PCA operates by identifying the most important directions in which data varies. When applied to image data, it enables compression by selecting the components that carry the most information, thereby allowing the reconstruction of the image using fewer components than the original data. This reduces the storage requirements while maintaining image quality, with some acceptable loss depending on the level of compression.

2 Problem Statement

The objective of this report is to investigate the use of PCA for image compression and reconstruction using Python. The primary goal is to reduce the size of image data while maintaining the ability to reconstruct the original images with acceptable quality. By applying PCA to image matrices, we aim to explore how varying the number of principal components impacts the quality of the reconstructed images. This study will also analyze the trade-off between the degree of data compression and the resulting image fidelity, demonstrating PCA's effectiveness as a dimension reduction technique in image processing tasks.

3 PCA Overview: Mathematical Foundation

PCA is a technique that transforms a large set of correlated variables into a smaller set of uncorrelated variables, known as Principal Components (PCs). The aim of PCA is to reduce the dimensionality of the data while retaining as much of the variability (information) as possible. This is done by finding a linear transformation that projects the data from a high-dimensional space to a lower-dimensional space, while maximizing the variance of the transformed data.

PCA works by creating new variables (PCs) that are linear combinations of the original variables. These PCs are uncorrelated and are ordered in such a way that the first few PCs capture most of the variability in the data. Mathematically, this is achieved through eigenvalue decomposition of the covariance matrix of the original data. The eigenvectors define the directions of the new axes (the PCs), and the eigenvalues indicate the amount of variance explained by each PC.

In matrix form, if X is a matrix of original variables, PCA finds a transformation matrix A such that the new variables $Y = AX$ are uncorrelated. The covariance matrix of Y , denoted by Σ_Y , is diagonal, meaning the variables in Y are uncorrelated.

3.1 Steps in PCA

3.1.1 Standardization of Data

PCA is sensitive to the scales of variables, meaning that variables measured in different units can disproportionately influence the results. To avoid this issue, it is common practice to standardize the data so that each variable has a mean of 0 and a variance of 1. This ensures that each variable contributes equally to the analysis. Standardization is particularly important when using the correlation matrix for PCA, as the correlation matrix implicitly standardizes the data by focusing on the relationships between variables, rather than their absolute values [3].

3.1.2 Covariance Matrix Computation

Once the data is standardized, the covariance matrix is computed. The covariance matrix summarizes how the variables change together. The elements of the covariance matrix

represent the covariances between pairs of variables. If the variables are highly correlated, PCA can find a linear combination of them to reduce redundancy. Mathematically, the covariance matrix Σ_X is computed as:

$$\Sigma_X = \frac{1}{n-1} X^T X$$

where X is the standardized data matrix, and n is the number of observations. [4]

3.1.3 Eigenvalue Decomposition

The next step is to perform eigenvalue decomposition on the covariance matrix. This results in eigenvalues and eigenvectors:

- Eigenvectors define the directions of the new principal component axes.
- Eigenvalues represent the amount of variance explained by each principal component.

The transformation matrix A is formed from the eigenvectors, and the eigenvalues indicate how much variance each PC captures. The principal components are uncorrelated because the eigenvectors are orthogonal to each other. The decomposition can be written as:

$$\Sigma_X = P \Lambda P^T$$

where P is the matrix of eigenvectors and Λ is the diagonal matrix of eigenvalues.

3.1.4 Selection of Components Based on Explained Variance

Not all principal components are needed to represent the data. The eigenvalues help in deciding how many components to retain. The explained variance is the proportion of total variance in the data that is captured by each PC. This is calculated as:

$$\text{Explained variance for PC} = \frac{\lambda_i}{\sum \lambda}$$

where λ_i is the eigenvalue corresponding to the i -th PC.

A scree plot is often used to visualize the eigenvalues and decide how many components to retain. Typically, the first few components that explain the majority of the variance are selected, while the rest are discarded.

4 PCA Workflow in Image Compression

The process of Principal Component Analysis (PCA) can be effectively used to compress images while retaining most of the critical information. As shown in Figure 1, the following steps outline how PCA is applied to an image matrix to reduce dimensionality:

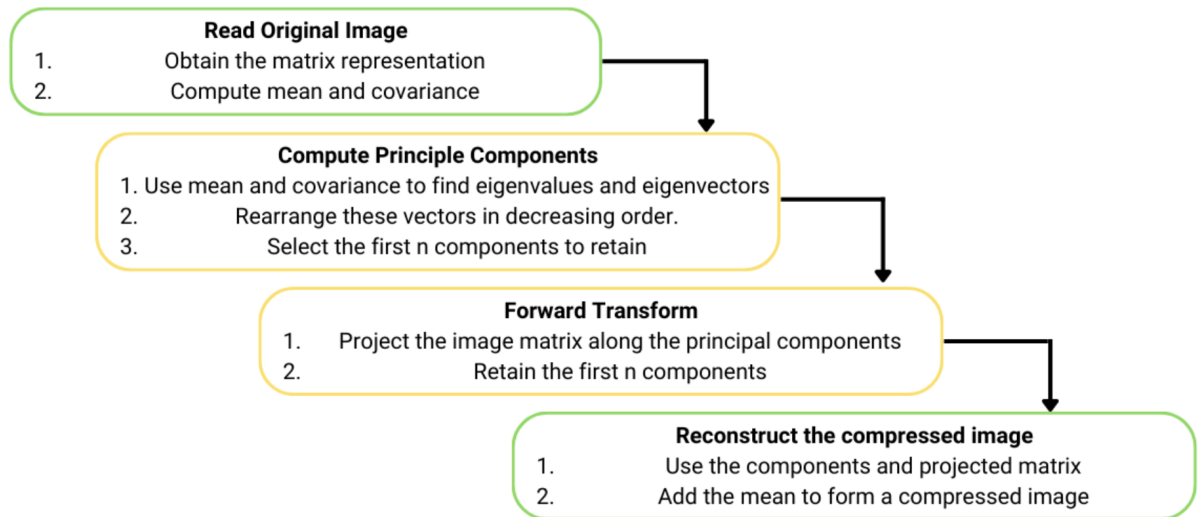


Figure 1: PCA Workflow in Image Compression Source [2].

A step-by-step explanation of how this process unfolds is presented in the following sections.

4.1 Read Original Image

Obtain the matrix representation: An image is essentially a matrix of pixel values. For grayscale images, this matrix has dimensions $m \times n$, where m is the number of rows (height) and n is the number of columns (width). Each element of the matrix represents a pixel intensity, typically ranging from 0 (black) to 255 (white).

For colored images, the image is typically represented as three separate matrices (for Red, Green, and Blue channels), but PCA can be applied separately to each channel.

Compute Mean and Covariance: For PCA, the mean and covariance of the image matrix need to be calculated. The mean helps center the data. Centering means subtracting the mean value from each element in the matrix, which shifts the data so that it is centered around zero, and the covariance helps understand how the variables (pixels) relate to each other.

4.2 Compute Principal Components

Find eigenvalues and eigenvectors: Using the covariance matrix, eigenvalues and eigenvectors are computed. Eigenvectors represent the directions of maximum variance, and eigenvalues show the magnitude of the variance in those directions.

Rearrange eigenvectors: Eigenvectors are sorted based on their eigenvalues in descending order, so the first few eigenvectors capture the most variance in the data. The higher the eigenvalue, the more important the corresponding eigenvector is in explaining the variability.

Select the first n components to retain: From the sorted eigenvectors, The top n eigenvectors with the highest eigenvalues are selected, which reduces the dataset's dimensionality while retaining most of its variance. The goal is to simplify the dataset by keeping only the most important components, minimizing information loss.

4.3 Forward Transform

Project the image matrix along the principal components: In this step, the original image data is transformed by projecting it onto the selected principal components. This projection essentially expresses the image in terms of the principal components, significantly reducing the number of dimensions.

Retain the first n components: Only the projections onto the top n components are kept, discarding the rest, which results in data compression.

4.4 Reconstruct the Compressed Image

Use the components and projected matrix: To reconstruct the compressed image, the original image is approximated by using the retained principal components and their projections. This reconstruction will not be identical to the original image due to the loss of information in discarded components, but it will be a close approximation.

Add the mean to form the compressed image: Since the data was originally centered by subtracting the mean, the final step is to add the mean back to the reconstructed matrix to produce the compressed image. The result is a compressed version of the original image, retaining the most important features but with reduced data.

5 Code Implementation of PCA in Python

5.1 Import Necessary Libraries

NumPy is used for array manipulation and numerical computations.

Matplotlib is for plotting images and visualizing the results.

PIL (Python Imaging Library) is used to load and process the image.

sklearn.decomposition.PCA is used to apply PCA for dimensionality reduction.

5.2 Load and Preprocess the Image

To load and preprocess the image, we begin by loading the image using the `Image.open()` function from the Python Imaging Library (PIL). Once the image is loaded, it is converted to grayscale using `.convert('L')`, which simplifies the processing by reducing the image to a single channel. The grayscale image is then transformed into a NumPy array, making it easier to manipulate the pixel data. In this array (`img_data`), each element represents a pixel's intensity, while `img` holds the original grayscale image for reference.

5.3 Apply PCA to the Image Data

The function `apply_pca` is used to apply Principal Component Analysis (PCA) to image data, reducing its dimensionality while preserving most of its variance. The image data is first flattened from 2D into 1D because PCA requires 1D data to operate. PCA is then performed by retaining a specified number of components (`n_components`), which determines the reduced dimensionality. The output includes:

- **n_components:** The number of principal components to retain for dimensionality reduction.
- **transformed_data:** The reduced representation of the image in the lower-dimensional space.
- **explained_variance:** The proportion of variance captured by each of the retained principal components, indicating how well the reduced data represents the original image.

5.4 Reconstruct the Image

After applying PCA, the image is reconstructed using the inverse transformation, which maps the reduced data back into the original feature space, although now represented with fewer components. This process results in an approximation of the original image, with some information potentially lost depending on how many components were retained. The reconstructed image is then converted back into a NumPy array with integer values, representing the pixel intensities, typically within the range of 0 to 255 for grayscale images.

reconstructed_data: Reconstructed image data in grayscale, reduced by PCA.

6 Result & Analysis

In the figure 2, the image on the left represents the original image, and the image on the right represents the reconstructed image after applying PCA, retaining 97% of the original variance.



Figure 2: Original Image vs Reconstructed Image ($n_components = 50$)

As the variance retained by PCA increases, the quality of the reconstructed image improves. In this case, with 97% of the variance retained, the image quality is close to the original.

The figure 3 shows the results of applying PCA to compress an image using different numbers of principal components (20, 40, 80, and 120). With 20 components, the image is highly distorted with significant loss of detail. At 40 components, the image quality improves but still shows distortion. By 80 components, most of the important details and



Figure 3: Reconstructed images using PCA with 20, 40, 80, and 120 principal components

structure are preserved, though minor distortion remains. At 120 components, the image is nearly identical to the original, with minimal distortion.

The variance explained by each PC is shown in a PCA's Scree plot in the below figure 4.

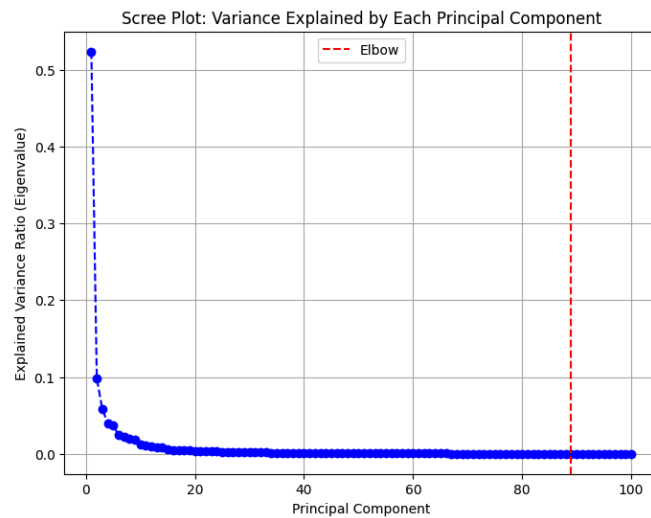


Figure 4: Variance Explained by Each Principal Component

The first few PC (the ones on the left) explain a significant amount of variance. After about 5-10 components, the curve flattens, indicating that the later components explain much less variance. The elbow point (marked in red) suggests that retaining around 80

components would capture the most significant variance in the dataset.

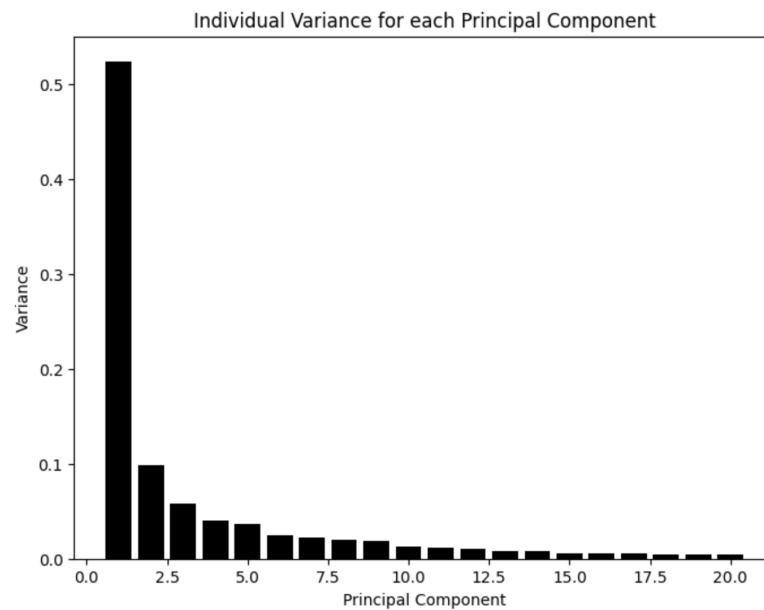


Figure 5: Individual Variance for each Principal Component

The figure above 5 suggests that most of the important variance in the dataset is captured by the first few components (1 to 5). The first PC explains the most variance over 50%, as indicated by the tallest bar. The second PC explains significantly less variance, followed by even smaller contributions from subsequent components.

7 Conclusion

PCA offers a powerful approach for dimensionality reduction, with numerous real-world applications in areas such as image compression, face recognition, and data visualization. By capturing the most significant variations in data, PCA helps reduce complexity, optimize storage, and improve processing efficiency. However, its effectiveness comes with trade-offs. PCA may lead to a loss of image quality, as lower-ranked components are discarded. Additionally, it is sensitive to the scaling of input variables, requiring careful preprocessing to ensure meaningful results. Furthermore, for large datasets, PCA can become computationally expensive, posing challenges in terms of processing time and resource usage.

References

- [1] I. T. Jolliffe. *Principal Component Analysis*. Springer Series in Statistics, 1986.
<https://doi.org/10.1007/978-1-4757-1904-8>
- [2] Ravish Raj. *Image Compression Using PCA*. EnjoyAlgorithms, 2024. <https://www.enjoyalgorithms.com/blog/image-compression-using-pca>
- [3] Jolliffe, I. T., & Cadima, J. (2016). Principal component analysis: A review and recent developments. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 374(2065), 20150202. <https://doi.org/10.1098/rsta.2015.0202>.
- [4] Jolliffe, I. T. (2002). *Principal Component Analysis (2nd ed.)*. Springer. <https://doi.org/10.1007/b98835>.